

ЛЕКЦИЯ 15

Реальное время и вычислительная архитектура комплекса

Реальное время — доказанная гарантия, а не выбор операционной системы

две части по 45

ение робототехническими комплексами»

План занятия

1

Три вида реального времени и типовая ошибка классификации

2

Модель периодических задач и загрузка

3

RM: оптимальность и граница Лю–Лейланда

4

Точный анализ времени отклика

5

EDF и поведение при перегрузке

6

Инверсия приоритетов и случай Mars Pathfinder

7

Протокол предельного приоритета

8

WCET: измерение против статического анализа

9

Сквозная задержка контура: разбор для примера A

10

Распределение функций и сеть комплекса

Синие — первая часть · янтарные — вторая часть

ЧАСТЬ 1

Планируемость: доказательство вместо надежды

Модель задач, планировщики со статическими и динамическими приоритетами, точный анализ.

15.1 Три вида реального времени

Вид	Последствие пропуска	Пример в РТК
Жёсткое (hard)	отказ системы, возможна авария	контур привода, аварийный останов
Твёрдое (firm)	результат бесполезен, но не опасен	кадр для визуального сервоуправления
Мягкое (soft)	снижение качества	интерфейс оператора, планирование миссии

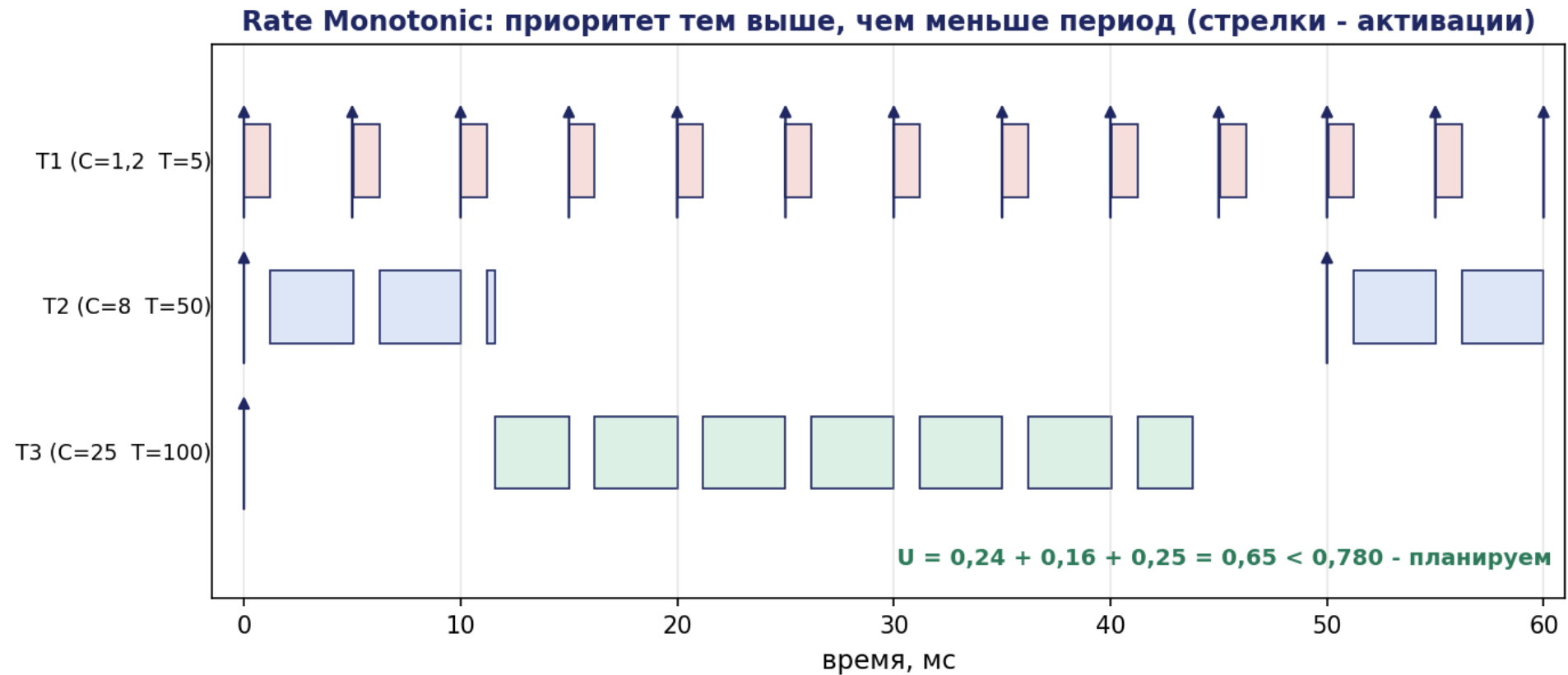
Отнесение всех задач к жёсткому реальному времени — проектная ошибка

$$\tau_i = (C_i, T_i, D_i), \quad U = \sum C_i / T_i$$

- C_i — наихудшее время выполнения (WCET)
- T_i — период; D_i — относительный дедлайн
- **Необходимое** условие планируемости: $U \leq 1$
- Спорадическая задача заменяется периодической с $T = T^{\min}$

Условие необходимо, но не достаточно: при статических приоритетах система может быть непланируема при $U < 1$

15.2 Планирование по Rate Monotonic



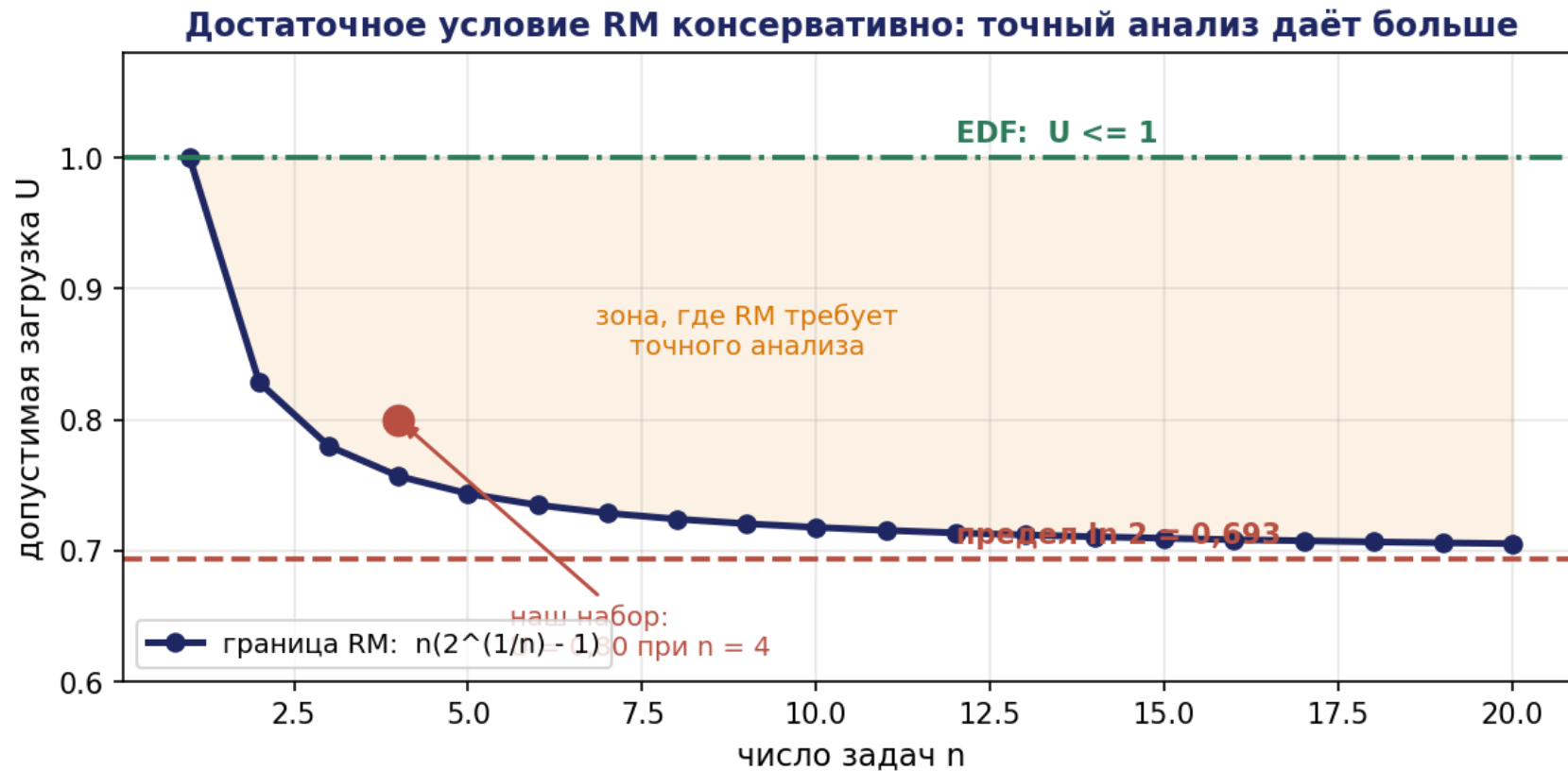
Приоритет тем выше, чем меньше период. Стрелки - моменты активации.

$$U \leq n \cdot (2^{1/n} - 1) \rightarrow \ln 2 \approx 0,693$$

- Достаточное условие планируемости при RM
- $n = 2$: 0,828; $n = 3$: 0,780; $n = 4$: 0,757; $n \rightarrow \infty$: 0,693
- При большом числе задач треть процессора резервируется под гарантию
- Условие достаточно, но не необходимо

RM оптимален среди статических приоритетов: приоритет тем выше, чем меньше период

15.3 Граница допустимой загрузки



Достаточное условие консервативно: при $U = 0,80$ и $n = 4$ точный анализ показывает планируемость.

15.4 Точный анализ времени отклика

$$R_i = C_i + \sum_{j < i} \lceil R_i / T_j \rceil \cdot C_j$$

- Наименьшая неподвижная точка; итерация от $R = C_i$
- Потолок считает число активаций более приоритетных задач за время R_i
- Последовательность монотонно не убывает, сходится при $U < 1$
- Планируемость $\Leftrightarrow R_i \leq D_i$ для всех задач

Необходимое и достаточное условие — в отличие от границы Лю–Лейланда

15.5 Разбор: четыре задачи бортового вычислителя

0,80

загрузка — выше границы 0,757
для $n=4$

11,6 мс

R_2 — обработка лидара,
дедлайн 50

43,8 мс

R_3 — планировщик траектории,
дедлайн 100

93,8 мс

R_4 — восприятие, дедлайн 200

- Достаточное условие не выполнено, а набор планируем
- $R_4 = 93,8$ мс при $C_4 = 30$ мс — время отклика втрое больше времени выполнения

? Проверим

Набор задач имеет $U = 0,85$ и не проходит достаточное условие RM. Что делать?

A Уменьшить загрузку до 0,757 — убрать задачу

B Перейти на EDF — он даёт $U \leq 1$

C Выполнить точный анализ времени отклика

D Поставить более быстрый процессор

? Проверим

Набор задач имеет $U = 0,85$ и не проходит достаточное условие RM. Что делать?

A Уменьшить загрузку до 0,757 — убрать задачу

B Перейти на EDF — он даёт $U \leq 1$

C Выполнить точный анализ времени отклика

D Поставить более быстрый процессор

Ответ. Третий: достаточное условие консервативно, набор может быть планируем. Точный анализ занимает минуты и даёт окончательный ответ.

15.6 RM против EDF

Rate Monotonic

- Статические приоритеты по периодам
- Граница $0,693...1,0$
- Низкие накладные расходы
- **Предсказуемая деградация**: страдают низкоприоритетные
- Повсеместно в промышленных RTOS

Earliest Deadline First

- Динамические приоритеты по дедлайнам
- Оптимален, граница $U \leq 1$
- Дороже: сортировка по дедлайнам
- **Эффект домино** при перегрузке
- Реже реализован

П Е Р Е Р Ы В

Конец первой части

Мы научились доказывать планируемость. После перерыва — что разрушает эти гарантии и как построить бюджет контура.

ЧАСТЬ 2

Что разрушает гарантии

Инверсия приоритетов, WCET, сквозная задержка и архитектура вычислительной системы.

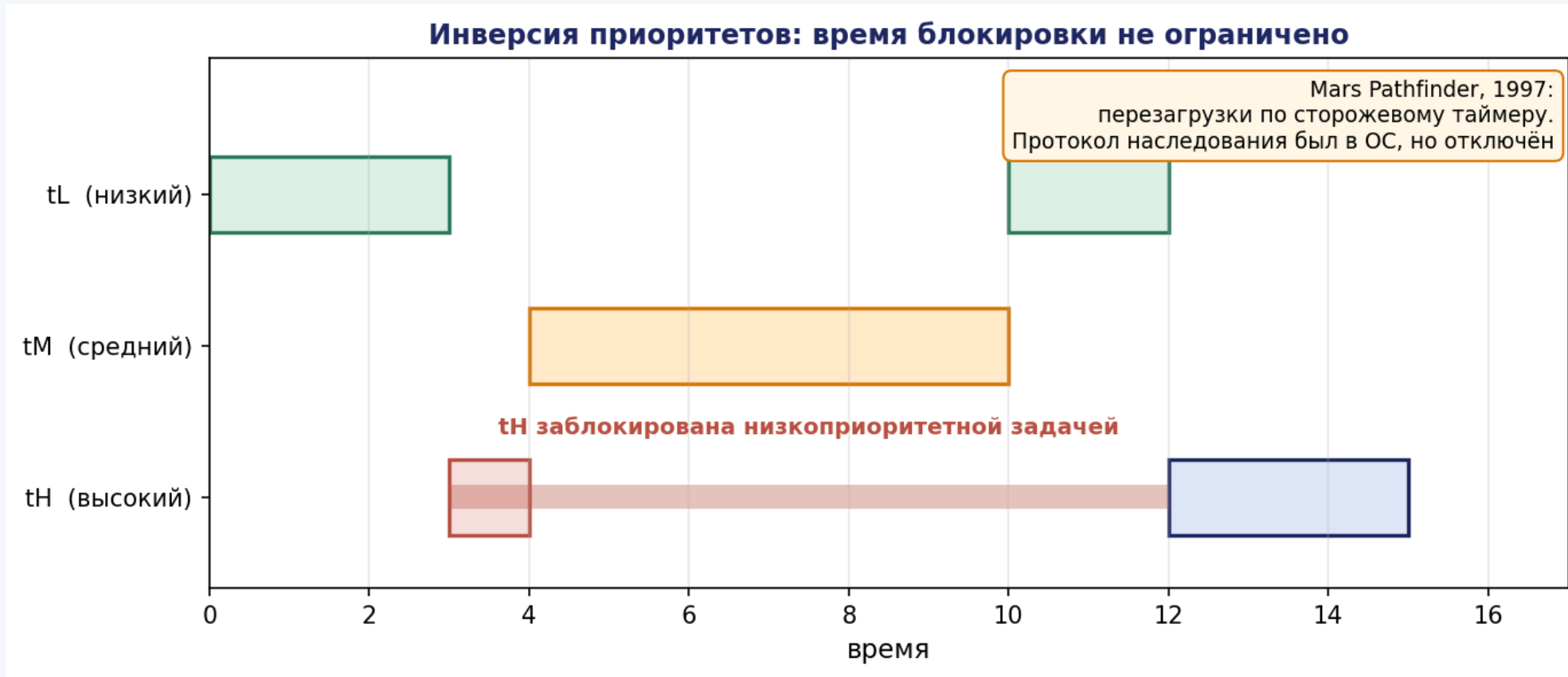
15.7 Инверсия приоритетов



Mars Pathfinder, 1997: перезагрузки по сторожевому таймеру из-за инверсии на шине VxWorks

15.7

Инверсия приоритетов



Mars Pathfinder, 1997: протокол наследования был в ОС, но отключён при конфигурировании.

Захват возможен, только если приоритет выше всех потолков занятых ресурсов

- Задача блокируется не более одного раза за активацию
- Время блокировки $\leq$ длительности наибольшей критической секции
- **Тупики невозможны**
- Доказательство: правило индуцирует строгий порядок на ресурсах

Тот же приём упорядочивания ресурсов, что в главе 4 — другая область применения

15.9 Учёт блокировки и практический вывод

- $R_i = C_i + B_i + \sum [R_i/T_j] \cdot C_j$
- B_i — наихудшее время блокировки, ограничено по теореме
- Длинная критическая секция низкоприоритетной задачи входит в отклик самой приоритетной
- Правило: в критической секции только операции с разделяемыми данными
- Вычисления, ввод-вывод и вызовы ядра — наружу

Факт

Критическая секция на 50 мс в фоновой задаче журналирования добавляет 50 мс к времени отклика контура управления приводом. Ни один профилировщик этого не покажет — задача-то фоновая.

15.10 Что разрушает предсказуемость времени выполнения

Фактор	Влияние	Мера
Кэш инструкций и данных	разброс в разы	блокировка кэша, разделение
Динамическое выделение памяти	неограниченные задержки	статические пулы
Сборка мусора	паузы в десятки мс	не применять в жёстких контурах
Виртуальная память	непредсказуемые задержки	mlockall
Разделяемый кэш на многоядерных	межъядерная интерференция	резервирование ядер

Последняя строка — главная нерешённая проблема современных платформ

15.11 WCET: три подхода

- **Измерение:** просто, но **не даёт гарантии** — наихудший путь мог не выполниться
- Инженерное соглашение: запас 20–50 % к измеренному максимуму
- **Статический анализ:** граф потока управления, модель процессора, IPET
- Даёт гарантированную оценку сверху, но пессимистичен
- **Гибрид:** измерения базовых блоков плюс статическая композиция

Ключевая идея

Практика для РТК: жёсткие контуры — статический анализ или измерения с большим запасом; твёрдые — измерения с запасом 30 % и мониторинг в эксплуатации; мягкие — таймаут и обработка превышения.

15.12 Сквозная задержка контура

$$L_{\{e2e\}} \leq \Sigma (T_i + R_i)$$

- T_i — наихудшее ожидание следующей активации задачи
- R_i — время отклика самой задачи
- При синхронной передаче слагаемые T_i исчезают: $L \leq \Sigma R_i$
- Различают **время реакции** и **возраст данных**

Событийная архитектура даёт выигрыш, но требует ограничения частоты событий

15.13 Разбор: контур визуальной доводки примера А

313 мс

сквозная задержка при
асинхронной передаче

47 мм

погрешность позиционирования
при 0,15 м/с

15 мм

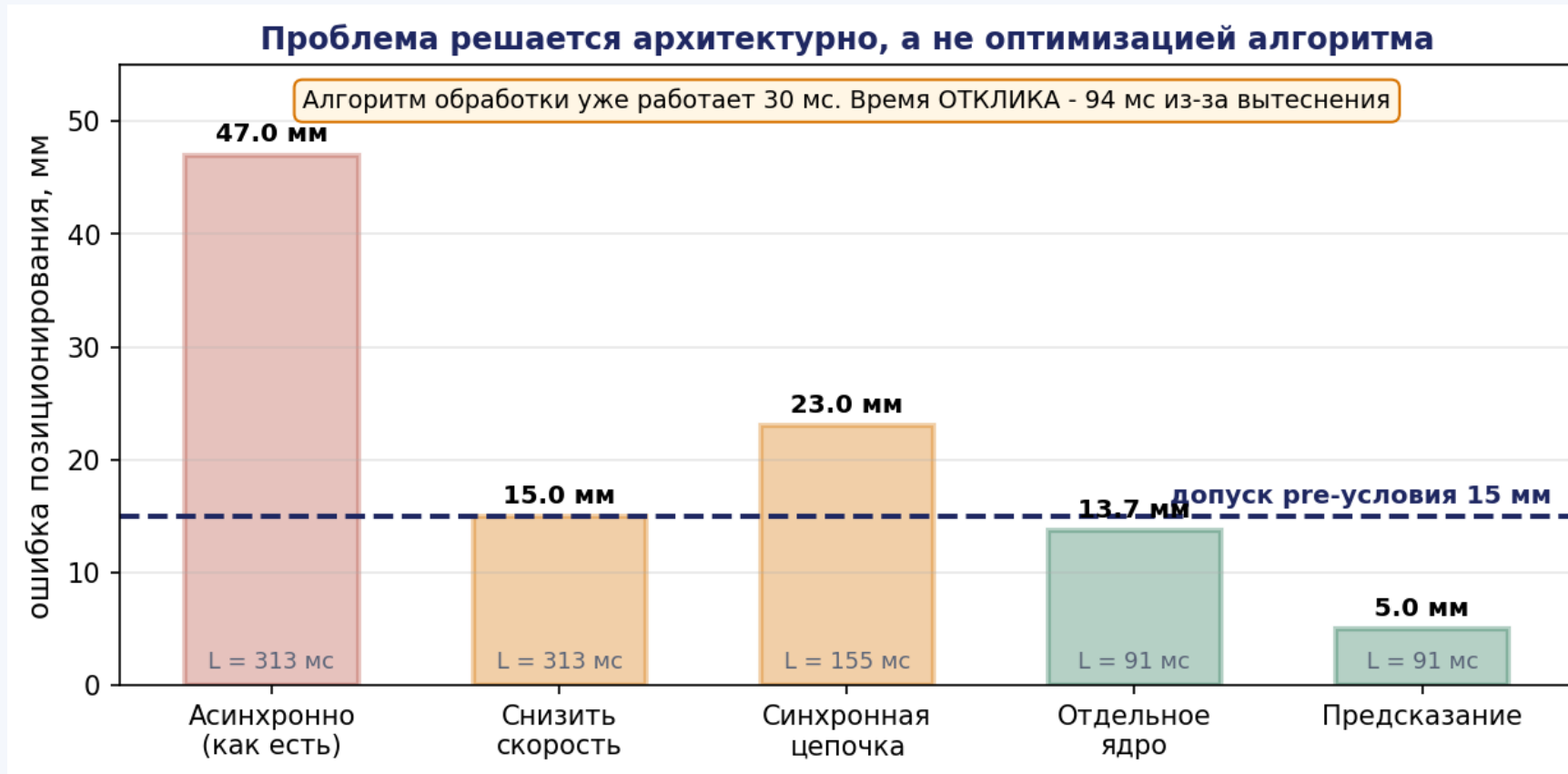
допуск из предусловия навыка

205 мс

наивная оценка из главы 1

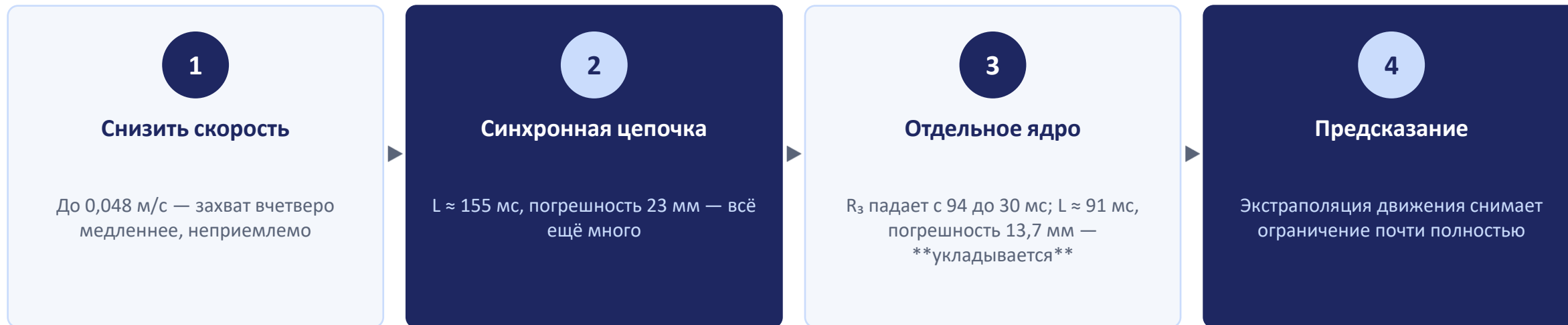
- Точный анализ даёт задержку в полтора раза больше наивной
- Требование нарушено втрое

15.13 Четыре решения проблемы задержки



Проблема решается размещением задачи на отдельном ядре, а не оптимизацией алгоритма.

15.14 Четыре решения и выбор



Проблема решается архитектурным решением, а не оптимизацией алгоритма

15.15 Распределение функций по вычислителям

Уровень	Что размещается	Требования
Привод	токовый и скоростной контуры	жёсткое РВ, 1–10 кГц
Борт робота	положение, локализация, защитный фильтр	твёрдое РВ, 100–1000 Гц
Ячейка	супервизор, координация	твёрдое РВ, 10–100 Гц
Цех	диспетчеризация, расписание, MES	мягкое РВ, 0,1–1 Гц
Предприятие	ERP, аналитика, обучение моделей	без РВ

Соответствует иерархии уровней управления из главы 1

15.16 Пять правил размещения

- **Замкнутый контур не пересекает ненадёжную границу** — отказ сети означает потерю управления
- **Защитные функции максимально низко** — аппаратно или на уровне привода
- **Выше уровень — больше горизонт, меньше частота**
- **Вычислительно тяжёлое и некритичное — выше**
- **Обработка ближе к источнику данных** — передавать детекции, а не видеопоток

Ключевая идея

Первое правило нарушается чаще всего: контур управления, замкнутый через Wi-Fi, работает превосходно до первого разрыва связи.

15.17 Сеть комплекса и QoS

- Обычная коммутируемая Ethernet не даёт гарантий: джиттер в десятки мс при коллизиях
- Детерминизм: **TSN** (802.1Qbv, 802.1AS), EtherCAT, PROFINET IRT
- DDS QoS: Reliability, Durability, History, **Deadline**, **Liveliness**
- Deadline и Liveliness реализуют мониторинг предположений из главы 9
- BEST_EFFORT для потоков сенсоров, RELIABLE для команд

Факт

Использование RELIABLE для видеопотока — распространённая ошибка: повторная передача устаревшего кадра бесполезна и лишь увеличивает задержку при потерях.

15.18 Синхронизация времени

- NTP: единицы–десятки миллисекунд — достаточно для журналирования
- PTP (IEEE 1588): единицы микросекунд при аппаратной поддержке
- PTP необходим для слияния данных лидара и камеры
- **Метка времени ставится в момент измерения, а не публикации**
- И сопровождает данные по всей цепочке обработки

Ключевая идея

Слияние данных по времени публикации даёт систематическую ошибку: разные датчики имеют разные задержки обработки. Ошибка в 50 мс при скорости 1 м/с — это 5 см смещения.

? Диагностируем

Робот устойчиво едет в симуляции и раскачивается на реальном роботе при той же настройке регулятора. Наиболее вероятная причина?

A Неточная модель динамики

B Неучтённая задержка контура

C Шум датчиков

D Ошибка в реализации регулятора

? Диагностируем

Робот устойчиво едет в симуляции и раскачивается на реальном роботе при той же настройке регулятора. Наиболее вероятная причина?

A Неточная модель динамики

B Неучтённая задержка контура

C Шум датчиков

D Ошибка в реализации регулятора

Ответ. Вторая. Задержка вносит фазовое запаздывание, снижающее запас устойчивости. В симуляции задержки обычно нулевые, на роботе — десятки миллисекунд.

“

«Алгоритм работает 30 миллисекунд» и «результат готов через 30 миллисекунд» — разные утверждения.

Между ними — вытеснение, ожидание активации и блокировка

Выводы

- 1 Реальное время — доказанная гарантия, а не свойство ОС; отнесение всех задач к жёсткому РВ — ошибка
- 2 $U \leq 1$ необходимо, но не достаточно; RM оптимален среди статических приоритетов
- 3 Точный анализ времени отклика даёт необходимое и достаточное условие
- 4 EDF оптимален, но катастрофичен при перегрузке; для критичных систем предпочтителен RM
- 5 Инверсия приоритетов разрушает все гарантии; PCP ограничивает блокировку и исключает тупики
- 6 WCET на многоядерных платформах с разделяемым кэшем — открытая проблема
- 7 Сквозная задержка может втрое превышать наивную оценку и решается архитектурно
- 8 Контур не пересекает ненадёжную границу; метка времени ставится в момент измерения

Вопросы для самопроверки

1. Почему выбор RTOS не является гарантией реального времени?
2. Классифицируйте: контур тока привода, распознавание детали, панель оператора, аварийный останов
3. Докажите необходимость условия $U \leq 1$
4. Почему граница Лю–Лейланда убывает с ростом числа задач?
5. Выведите уравнение времени отклика; каков смысл функции потолка?
6. Почему EDF катастрофичен при перегрузке, а RM нет?
7. Опишите механизм инверсии приоритетов и случай Mars Pathfinder
8. Как PCP исключает тупики? С каким результатом главы 4 это связано?
9. Почему измеренное время алгоритма не равно времени отклика? Приведите пример
10. Почему метка времени ставится в момент измерения, а не публикации?